

beginning

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

data loading

```
df_encoded = pd.read_excel("/content/drive/MyDrive/capstone/datasets/5_encoded.xlsx")
```

```
df_encoded.head()
```

	age	latitude	longitude	feels_like	temp_min	temp_max	air_pressure	air_humidity	wind_speed	wind_deg	...	gender_encoded_male	gender_encoded_thir gende
0	NaN	23.960600	91.119089	302.86	299.15	299.15	1008.0	83.0	2.10	320.0	...	1.0	0.
1	20.0	24.899220	91.868527	299.17	295.57	295.57	1012.0	96.0	1.39	116.0	...	1.0	0.
2	14.0	23.832984	89.966688	304.33	301.15	301.15	1004.0	78.0	3.60	190.0	...	0.0	0.
3	NaN	24.850066	89.372843	303.67	299.34	299.34	1003.0	91.0	2.64	186.0	...	0.0	0.
4	18.0	23.764386	90.389014	304.49	300.15	300.15	1004.0	83.0	1.99	154.0	...	0.0	0.

5 rows × 37 columns

```
df_encoded.shape
```

```
(1617, 37)
```

```
def missing_data_report(df):
    # Calculate the percentage of missing data for each column
    missing_percentage = df.isna().mean() * 100

    # Get the data types of each column
    data_types = df.dtypes

    # Create a DataFrame for the missing data summary including data types
    missing_data_summary = pd.DataFrame({
        'Column': missing_percentage.index,
        'Missing Percentage': missing_percentage.values,
        'Data Type': data_types.values
    })

    # Define the four groups based on missing percentage
    group_1 = missing_data_summary[missing_data_summary['Missing Percentage'] > 70][['Column', 'Missing Percentage', 'Data Type']]
    group_2 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 40) & (missing_data_summary['Missing Percentage'] <= 70)][['Column', 'Missing Percentage', 'Data Type']]
    group_3 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 20) & (missing_data_summary['Missing Percentage'] <= 40)][['Column', 'Missing Percentage', 'Data Type']]
    group_4 = missing_data_summary[missing_data_summary['Missing Percentage'] <= 20][['Column', 'Missing Percentage', 'Data Type']]

    # Sort each group by missing percentage in descending order (most missing to least missing)
    group_1 = group_1.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
    group_2 = group_2.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
    group_3 = group_3.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
    group_4 = group_4.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)

    # Rename columns for clarity
    group_1.columns = ['Column', 'Missing %', 'Data Type']
    group_2.columns = ['Column', 'Missing %', 'Data Type']
    group_3.columns = ['Column', 'Missing %', 'Data Type']
    group_4.columns = ['Column', 'Missing %', 'Data Type']

    # Display each group one by one, from most missing to least missing
    print("Group 1: Columns with >70% missing data")
    print(group_1.to_string(index=False))
    print("\nGroup 2: Columns with >40% to ≤70% missing data")
    print(group_2.to_string(index=False))
    print("\nGroup 3: Columns with >20% to ≤40% missing data")
    print(group_3.to_string(index=False))
    print("\nGroup 4: Columns with ≤20% missing data")
    print(group_4.to_string(index=False))
```

```
missing_data_report(df_encoded)
```

```
Group 1: Columns with >70% missing data
Empty DataFrame
Columns: [Column, Missing %, Data Type]
Index: []
```

```

Group 2: Columns with >40% to ≤70% missing data
      Column  Missing %  Data Type
      feels_like  53.061224  float64
      air_humidity  53.061224  float64
      wind_deg  53.061224  float64
      clouds_sky  53.061224  float64
      weather_main_encoded  52.999382  float64
      weather_description_encoded  52.999382  float64
      financial_condition_encoded  45.516388  float64

```

```

Group 3: Columns with >20% to ≤40% missing data
      Column  Missing %  Data Type
      workplace_encoded_semi-urban  37.909709  float64
      workplace_encoded_urban  37.909709  float64
      workplace_encoded_rural  37.909709  float64
      air_pressure  30.426716  float64
      wind_speed  30.179344  float64
      temp_min  29.870130  float64
      temp_max  29.560915  float64
      marital_status_encoded  23.933210  float64

```

```

Group 4: Columns with ≤20% missing data
      Column  Missing %  Data Type
      profession_group_encoded  18.923933  float64
      reason_encoded  16.821274  float64
      age  12.739641  float64
      time_encoded  12.059369  float64
      age_group_encoded  7.421150  float64
      latitude  6.493506  float64
      longitude  6.493506  float64
      method_encoded  2.906617  float64
      suicide_month  2.411874  float64
      suicide_day  2.411874  float64
      suicide_weekday_encoded  2.411874  float64
      suicide_week  2.411874  float64
      suicide_year  2.411874  float64
      religion_encoded_buddhist  2.411874  float64
      religion_encoded_muslim  2.411874  float64
      religion_encoded_indigenous  2.411874  float64
      religion_encoded_hindu  2.411874  float64
      religion_encoded_christian  2.411874  float64
      hometown_encoded  1.669759  float64
      gender_encoded_third_gender  0.432900  float64
      gender_encoded_female  0.432900  float64
      gender_encoded_male  0.432900  float64

```

```
df_encoded.columns
```

```

Index(['age', 'latitude', 'longitude', 'feels_like', 'temp_min', 'temp_max',
      'air_pressure', 'air_humidity', 'wind_speed', 'wind_deg', 'clouds_sky',
      'time_encoded', 'suicide_day', 'suicide_month', 'suicide_year',
      'suicide_week', 'age_group_encoded', 'financial_condition_encoded',
      'weather_main_encoded', 'suicide_weekday_encoded',
      'profession_group_encoded', 'hometown_encoded', 'reason_encoded',
      'method_encoded', 'weather_description_encoded',

```

```
'marital_status_encoded', 'gender_encoded_female',
'gender_encoded_male', 'gender_encoded_third gender',
'workplace_encoded_rural', 'workplace_encoded_semi-urban',
'workplace_encoded_urban', 'religion_encoded_buddhist',
'religion_encoded_christian', 'religion_encoded_hindu',
'religion_encoded_indigenous', 'religion_encoded_muslim'],
dtype='object')
```

```
import pandas as pd

def compute_sorted_correlations(df, threshold=0.0):
    corr_matrix = df.corr()

    result = []

    for col in corr_matrix.columns:
        # Drop self-correlation
        related = corr_matrix[col].drop(labels=[col])

        # Keep only correlations above threshold (excluding exact 0)
        related_filtered = related[related.abs() > threshold]

        # Sort by absolute correlation descending
        related_sorted = related_filtered.reindex(related_filtered.abs().sort_values(ascending=False).index)

        # Format results
        related_list = [
            {'related_column': other_col, 'related_value': round(related_value, 4)}
            for other_col, related_value in related_sorted.items()
        ]

        result.append({
            'column_name': col,
            'related_columns': related_list
        })

    return result
```

```
correlation_dict = compute_sorted_correlations(df_encoded)
```

```
from pprint import pprint
display(correlation_dict[:2]) # preview first 2 entries
```

[Show hidden output](#)

```
df_encoded.columns
```

```
Index(['age', 'latitude', 'longitude', 'feels_like', 'temp_min', 'temp_max',
       'air_pressure', 'air_humidity', 'wind_speed', 'wind_deg', 'clouds_sky',
       'time_encoded', 'suicide_day', 'suicide_month', 'suicide_year',
```

```
'suicide_week', 'age_group_encoded', 'financial_condition_encoded',
'weather_main_encoded', 'suicide_weekday_encoded',
'profession_group_encoded', 'hometown_encoded', 'reason_encoded',
'method_encoded', 'weather_description_encoded',
'marital_status_encoded', 'gender_encoded_female',
'gender_encoded_male', 'gender_encoded_third_gender',
'workplace_encoded_rural', 'workplace_encoded_semi-urban',
'workplace_encoded_urban', 'religion_encoded_buddhist',
'religion_encoded_christian', 'religion_encoded_hindu',
'religion_encoded_indigenous', 'religion_encoded_muslim'],
dtype='object')
```

```
from tqdm import tqdm
from sklearn.linear_model import BayesianRidge
from copy import deepcopy

# Start with a deep copy of the dataset so we don't affect the original
df_imputed_mice_context = deepcopy(df_encoded)
df_imputed_mice_no_context = deepcopy(df_encoded)

# === Define constraints ===
constraints = {
    'age': (0, 100),
    'latitude': (20.57, 26.63),          # Bangladesh latitude range
    'longitude': (88.0167, 92.6833),     # Bangladesh longitude range
    'feels_like': (2.6, 300.65),         # Temperature range in Kelvin (lowest and highest)
    'temp_min': (2.6, 300.65),          # Same temp range
    'temp_max': (2.6, 300.65),
    'air_pressure': (1000, 1050),        # ~1000 hPa to a bit above 1020 hPa
    'air_humidity': (70, 100),          # Percent
    'wind_speed': (3, 20),              # ~3 to 16+ km/h, added small buffer
    'wind_deg': (0, 360),               # Wind direction in degrees, full circle
    'clouds_sky': (0, 100),             # Cloudiness in percent
    'time_encoded': (0, 23),            # Assuming time encoded as hour in 24h format
    'suicide_day': (1, 31),             # Day of month
    'suicide_month': (1, 12),           # Month number
    'suicide_year': (2017, 2025),       # Year range you gave
    'suicide_week': (1, 52),           # Week number (approximate)
    'suicide_weekday_encoded': (0, 6),
    'age_group_encoded': (0, 6),
    'financial_condition_encoded': (0, 4),
    'weather_main_encoded': (0, 7),
}
```

```
def impute_column_context(column_name, context_info, df, constraints={}):
    """
    Impute missing values in column_name using only non-zero correlated columns as predictors.
    context_info should contain 'related_columns' list with dicts like:
    [{'related_column': 'col1'}, {'related_column': 'col2'}, ...]
    """
```

```
related_cols = [r['related_column'] for r in context_info['related_columns']]
missing_mask = df[column_name].isna()

imputed_count = 0
skipped_count = 0
errors = []

for idx in tqdm(df[missing_mask].index, desc=f"Imputing '{column_name}' (context)"):
    try:
        row = df.loc[idx]
        predictors = [col for col in related_cols if pd.notna(row[col])]

        if not predictors: # no usable predictors for this row
            skipped_count += 1
            continue

        # Prepare training data – only rows where target & predictors are non-null
        train_df = df.dropna(subset=[column_name] + predictors)
        if train_df.empty:
            skipped_count += 1
            continue

        X_train = train_df[predictors].values
        y_train = train_df[column_name].values

        model = BayesianRidge()
        model.fit(X_train, y_train)

        # Test-time: fill missing predictors with column means
        row_input = row[predictors].copy()
        for col in predictors:
            if pd.isna(row_input[col]):
                row_input[col] = df[col].mean()

        X_test = np.array(row_input).reshape(1, -1)
        predicted = model.predict(X_test)[0]

        # Apply constraints
        if column_name in constraints:
            lower, upper = constraints[column_name]
            if lower is not None:
                predicted = max(predicted, lower)
            if upper is not None:
                predicted = min(predicted, upper)

        df.at[idx, column_name] = predicted
        imputed_count += 1

    except Exception as e:
        errors.append((idx, str(e)))
```

```

print(f"\n'{column_name}' Imputation Summary (Context):")
print(f" - Imputed values: {imputed_count}")
print(f" - Skipped (no predictors): {skipped_count}")
print(f" - Errors: {len(errors)}")
if errors:
    print("Sample Errors:")
    for idx, err in errors[:5]:
        print(f" - Row {idx}: {err}")

```

```

# === Run imputation for each column ===
for col_info in correlation_dict:
    impute_column_context(col_info['column_name'], col_info, df_imputed_mice_context)

```

[Show hidden output](#)

```
missing_data_report(df_imputed_mice_context)
```

[Show hidden output](#)

```

def impute_column_no_context(column_name, df, constraints={}):
    """
    Impute missing values in column_name using all available non-null columns as predictors.
    """
    possible_predictors = [col for col in df.columns if col != column_name]
    missing_mask = df[column_name].isna()

    imputed_count = 0
    skipped_count = 0
    errors = []

    for idx in tqdm(df[missing_mask].index, desc=f"Imputing '{column_name}' (no context)":
        try:
            row = df.loc[idx]
            predictors = [col for col in possible_predictors if pd.notna(row[col])]

            if not predictors: # no usable predictors for this row
                skipped_count += 1
                continue

            # Prepare training data - only rows where target & predictors are non-null
            train_df = df.dropna(subset=[column_name] + predictors)
            if train_df.empty:
                skipped_count += 1
                continue

            X_train = train_df[predictors].values
            y_train = train_df[column_name].values

            model = BayesianRidge()

```

```

model.fit(X_train, y_train)

# Test-time: fill missing predictors with column means
row_input = row[predictors].copy()
for col in predictors:
    if pd.isna(row_input[col]):
        row_input[col] = df[col].mean()

X_test = np.array(row_input).reshape(1, -1)
predicted = model.predict(X_test)[0]

# Apply constraints
if column_name in constraints:
    lower, upper = constraints[column_name]
    if lower is not None:
        predicted = max(predicted, lower)
    if upper is not None:
        predicted = min(predicted, upper)

df.at[idx, column_name] = predicted
imputed_count += 1

except Exception as e:
    errors.append((idx, str(e)))

print(f"\n'{column_name}' Imputation Summary (No Context):")
print(f" - Imputed values: {imputed_count}")
print(f" - Skipped (no predictors): {skipped_count}")
print(f" - Errors: {len(errors)}")
if errors:
    print("Sample Errors:")
    for idx, err in errors[:5]:
        print(f" - Row {idx}: {err}")

```

```

# Run imputation without context for each column
for col in df_imputed_mice_no_context.columns:
    impute_column_no_context(col, df_imputed_mice_no_context, constraints=constraints)

```

```

Imputing 'age' (no context): 100%|██████████| 206/206 [00:01<00:00, 115.01it/s]

'age' Imputation Summary (No Context):
- Imputed values: 202
- Skipped (no predictors): 4
- Errors: 0
Imputing 'latitude' (no context): 100%|██████████| 105/105 [00:02<00:00, 41.25it/s]

'latitude' Imputation Summary (No Context):
- Imputed values: 101
- Skipped (no predictors): 4
- Errors: 0
Imputing 'longitude' (no context): 100%|██████████| 105/105 [00:01<00:00, 97.10it/s]

```



```

'longitude' Imputation Summary (No Context):
- Imputed values: 101
- Skipped (no predictors): 4
- Errors: 0
Imputing 'feels_like' (no context): 100%|██████████| 858/858 [00:04<00:00, 207.07it/s]

'feels_like' Imputation Summary (No Context):
- Imputed values: 854
- Skipped (no predictors): 4
- Errors: 0
Imputing 'temp_min' (no context): 100%|██████████| 483/483 [00:02<00:00, 185.80it/s]

'temp_min' Imputation Summary (No Context):
- Imputed values: 479
- Skipped (no predictors): 4
- Errors: 0
Imputing 'temp_max' (no context): 100%|██████████| 478/478 [00:02<00:00, 195.42it/s]

'temp_max' Imputation Summary (No Context):
- Imputed values: 474
- Skipped (no predictors): 4
- Errors: 0
Imputing 'air_pressure' (no context): 100%|██████████| 492/492 [00:05<00:00, 86.20it/s]

'air_pressure' Imputation Summary (No Context):
- Imputed values: 488
- Skipped (no predictors): 4
- Errors: 0
Imputing 'air_humidity' (no context): 100%|██████████| 858/858 [00:04<00:00, 181.99it/s]

'air_humidity' Imputation Summary (No Context):
- Imputed values: 854
- Skipped (no predictors): 4
- Errors: 0
Imputing 'wind_speed' (no context): 100%|██████████| 488/488 [00:02<00:00, 171.53it/s]

'wind_speed' Imputation Summary (No Context):
- Imputed values: 484
- Skipped (no predictors): 4
- Errors: 0
Imputing 'wind_deg' (no context): 100%|██████████| 858/858 [00:07<00:00, 110.91it/s]

'wind_deg' Imputation Summary (No Context):
- Imputed values: 854

```

```
missing_data_report(df_imputed_mice_no_context)
```

```

Group 1: Columns with >70% missing data
Empty DataFrame
Columns: [Column, Missing %, Data Type]
Index: []

```

```

Group 2: Columns with >40% to ≤70% missing data
Empty DataFrame
Columns: [Column, Missing %, Data Type]

```

```
Index: []
```

```
Group 3: Columns with >20% to ≤40% missing data
```

```
Empty DataFrame
```

```
Columns: [Column, Missing %, Data Type]
```

```
Index: []
```

```
Group 4: Columns with ≤20% missing data
```

Column	Missing %	Data Type
age	0.247372	float64
latitude	0.247372	float64
longitude	0.247372	float64
feels_like	0.247372	float64
temp_min	0.247372	float64
temp_max	0.247372	float64
air_pressure	0.247372	float64
air_humidity	0.247372	float64
wind_speed	0.247372	float64
wind_deg	0.247372	float64
clouds_sky	0.247372	float64
time_encoded	0.247372	float64
suicide_day	0.247372	float64
suicide_month	0.247372	float64
suicide_year	0.247372	float64
suicide_week	0.247372	float64
age_group_encoded	0.247372	float64
financial_condition_encoded	0.247372	float64
weather_main_encoded	0.247372	float64
suicide_weekday_encoded	0.247372	float64
profession_group_encoded	0.247372	float64
hometown_encoded	0.247372	float64
reason_encoded	0.247372	float64
method_encoded	0.247372	float64
weather_description_encoded	0.247372	float64
marital_status_encoded	0.247372	float64
gender_encoded_female	0.247372	float64
gender_encoded_male	0.247372	float64
gender_encoded_third gender	0.247372	float64
workplace_encoded_rural	0.247372	float64
workplace_encoded_semi-urban	0.247372	float64
workplace_encoded_urban	0.247372	float64
religion_encoded_buddhist	0.247372	float64
religion_encoded_christian	0.247372	float64
religion_encoded_hindu	0.247372	float64
religion_encoded_indigenous	0.247372	float64
religion_encoded_muslim	0.247372	float64

✓ After imputation processing

```
import numpy as np
```

```
# Define one-hot groups
```

```

one_hot_groups = {
    'gender': [
        'gender_encoded_female',
        'gender_encoded_male',
        'gender_encoded_third gender'
    ],
    'workplace': [
        'workplace_encoded_rural',
        'workplace_encoded_semi-urban',
        'workplace_encoded_urban'
    ],
    'religion': [
        'religion_encoded_buddhist',
        'religion_encoded_christian',
        'religion_encoded_hindu',
        'religion_encoded_indigenous',
        'religion_encoded_muslim'
    ]
}

# Function to fix one-hot encoding
def fix_one_hot_group(df, columns):
    data = df[columns].to_numpy()
    max_indices = np.argmax(data, axis=1) # index of max value in each row

    # Set the max column to 1, others to 0
    fixed_data = np.zeros_like(data)
    fixed_data[np.arange(data.shape[0]), max_indices] = 1

    df[columns] = fixed_data
    return df

```

✓ fixing one hot columns for context data

```

# Apply fix to each group
for group_cols in one_hot_groups.values():
    df_imputed_mice_context = fix_one_hot_group(df_imputed_mice_context, group_cols)

```

✓ fixing one hot columns for no context data

```

# Apply fix to each group
for group_cols in one_hot_groups.values():
    df_imputed_mice_no_context = fix_one_hot_group(df_imputed_mice_no_context, group_cols)

```

```

# Define your constraints
ordinal_constraints = {

```

```
'suicide_weekday_encoded': (0, 6),
'age_group_encoded': (0, 6),
'financial_condition_encoded': (0, 4),
'weather_main_encoded': (0, 7),
}
```

✓ fixing ordinal columns for context data

```
# Apply rounding and clip to valid range
for col, (low, high) in ordinal_constraints.items():
    if col in df_imputed_mice_context.columns:
        df_imputed_mice_context[col] = df_imputed_mice_context[col].round().clip(lower=low, upper=high)
```

✓ fixing ordinal columns for no context data

```
# Apply rounding and clip to valid range
for col, (low, high) in ordinal_constraints.items():
    if col in df_imputed_mice_context.columns:
        df_imputed_mice_no_context[col] = df_imputed_mice_no_context[col].round().clip(lower=low, upper=high)
```

```
df_imputed_mice_context.tail(5)
```

	age	latitude	longitude	feels_like	temp_min	temp_max	air_pressure	air_humidity	wind_speed	wind_deg	...	gender_encoded_male	gender_en
1612	15.000000	23.707123	90.067937	-16.939983	55.508111	65.034355	1010.215950	408.127411	6.177175	-577.892755	...	0.0	
1613	27.000000	23.740193	90.302765	143.521566	178.146784	180.409645	1002.507166	253.150332	3.510377	-226.057716	...	0.0	
1614	9.000000	23.825789	90.068352	203.686195	224.055094	226.010537	1004.803463	187.071699	4.021778	-86.706507	...	1.0	
1615	22.368872	23.898795	89.129554	-74.633303	12.500000	23.700000	1019.000000	457.222918	8.000000	-771.356660	...	0.0	
1616	26.000000	22.655478	89.794181	-49.798064	29.800000	36.600000	1003.800000	458.643328	3.800000	-656.035418	...	0.0	

5 rows × 37 columns

```
df_imputed_mice_no_context.tail(5)
```

	age	latitude	longitude	feels_like	temp_min	temp_max	air_pressure	air_humidity	wind_speed	wind_deg	...	gender_encoded_male	gender_encoded_female
1612	15.000000	23.707123	90.067937	119.627275	121.154277	129.013497	1009.870911	93.222221	5.302351	62.488973	...	0.0	0.0
1613	27.000000	23.740193	90.302765	128.211925	138.185201	141.419927	1002.570011	93.227106	4.001096	61.653333	...	0.0	0.0
1614	9.000000	23.825789	90.068352	166.734308	171.723086	174.944481	1004.899221	91.553503	4.760535	80.759772	...	1.0	0.0
1615	22.368872	23.898795	89.129554	3.628097	12.500000	23.700000	1019.000000	99.502018	8.000000	0.000000	...	0.0	0.0
1616	26.000000	22.655478	89.794181	15.713082	29.800000	36.600000	1003.800000	98.716658	3.800000	14.131072	...	0.0	0.0

5 rows × 37 columns

```
# # prompt: give me the code make the latest df downloadable and download that db named final_suicide_cleaned.xlsx ins excel format
```

```
# # Make the latest DataFrame downloadable
```

```
# from google.colab import files
```

```
# # Specify the filename for the Excel file
```

```
# filename1 = '6_mice_context.xlsx'
```

```
# # Save the DataFrame to an Excel file
```

```
# df_imputed_mice_context.to_excel(filename1, index=False)
```

```
# # Download the file
```

```
# files.download(filename1)
```

```
# # prompt: give me the code make the latest df downloadable and download that db named final_suicide_cleaned.xlsx ins excel format
```

```
# # Make the latest DataFrame downloadable
```

```
# from google.colab import files
```

```
# # Specify the filename for the Excel file
```

```
# filename2 = '7_mice_no_context.xlsx'
```

```
# # Save the DataFrame to an Excel file
```

```
# df_imputed_mice_no_context.to_excel(filename2, index=False)
```

```
# # Download the file
```

```
# files.download(filename2)
```

